

SRE Bot — Release 1 Functional Specification

(GitHub-flavored Markdown)

Release 1 theme: *Infrastructure & Platform Discovery via Chat* with the ability to **view / add / modify / delete** (CRUD) **service↔infrastructure associations** in a governed, auditable way.

Table of Contents

1. Problem & Goals
2. Personas
3. In-Scope for Release 1
4. Out of Scope (R2+)
5. Functional Requirements
6. Non-Functional Requirements
7. User Journeys
8. APIs & Data Model
9. Security, RBAC & Audit
10. Acceptance Criteria
11. Architecture Diagrams
 - User Interaction Sequence
 - High-level Architecture

- Component-level Architecture

12. Test Plan

13. Delivery & Ops

14. Epics & Stories for Jira

15. How to post Epics & Stories using a Jira MCP Server

Problem & Goals

Engineering teams (SRE, Platform, SecOps, Developers) need a single conversational interface to:

- Discover **what runs where** across multi-account **AWS** and **Kubernetes (EKS)**.
- Visualize and **correct associations** between *services/apps* and *infra components*.
- Export light-weight reports from chat (CSV/JSON) for fast sharing.

Goals (R1):

1. Answer discovery questions from **live inventory** and **cached** data.
2. Provide **CRUD** for *Service↔Infra* associations with validations and audit.
3. Export **tables** returned in chat to **CSV/JSON**.
4. Enforce **SSO + RBAC**, **read-only cloud access**, and **full audit** for writes.
5. Keep the **LLM pluggable** and responses grounded in retrieved context.

Personas

Persona

Needs

SRE / DevOps	“Where does payments-api run in prod?” “Show all EKS clusters in eu-west-1.”
Platform Engineer	Inventory by env; map services→namespaces→clusters; CSV export.
Security (SecOps)	Who owns workload X? What accounts/regions is it deployed in?
Developer / Service Owner	Quick answers to <i>where is my service</i> and <i>which infra it depends on</i> .
Product Owner / PM	One-line exports (matrix of service→env→cluster).

In-Scope for Release 1

- Chat-based **discovery queries** (inventory, topology, ownership).
 - **Association Store** and **CRUD** in chat/API.
 - **AWS** (accounts, regions, VPC, Subnets, EC2, RDS, **EKS**) **read-only**.
 - **Kubernetes** (clusters, namespaces, deployments, services) **read-only**.
 - Optional **CMDB/Service Catalog** for owners (read-only).
 - **CSV/JSON export** of any result set.
 - **SSO (OIDC/SAML), RBAC, Audit logging**.
-

Out of Scope (R2+)

- Automated remediation or recommendations.
 - Complex cost, performance optimizations.
 - Migration and deep network troubleshooting flows.
-

Functional Requirements

F1 — Conversational Discovery

- Query types (examples):
 - **Inventory**: “List all prod EKS clusters in eu-west-1.”
 - **Topology**: “What VPC and Subnets host `payments-api`?”
 - **Where-does-it-run**: “Where does `risk-engine` run across environments?”
 - **Ownership**: “Who owns `orders-api`?”
- Answers: natural language + table. User can request “**show JSON**” or “**export CSV**”.
- Bot clearly cites **accounts/regions** used and the **data sources**.

F2 — Association Management (CRUD)

- **Entities**:
 - **Service**: `{id, name, owner_team?, repo_url?, tags{}}`
 - **InfraComponent**: `{id, type, cloud, account_id?, region?, resource_arn?|k8s_uid?, name?, env?, tags{}}`
 - **Association**: `{id, service_id, infra_id, relation_type, env, created_by, created_at, updated_at, notes, confidence{source,score}}`
- **relation_type** $\in$ `{runs_on, depends_on, stores_in, connects_to, owned_by}`.
- Validation: IDs resolve; env in `{dev,test,stg,prod,...}`; relation_type valid; RBAC enforced.
- All writes need explicit confirmation in chat.

F3 — Exports

- Users can export any table to **CSV** or **JSON**.
- Exports are **named** and **audited**.

F4 — Inventory & Cache

- Scheduled refresh every ≤ 15 min per connected account/cluster.
- On-demand scoped refresh (role-gated; rate-limited).

Non-Functional Requirements

NFR	Target
Latency	$P50 \leq 4s$ (cached), $P95 \leq 12s$; forced refresh $\leq 30s$.
Availability	$\geq 99.5\%$ monthly for chat/API.
Security	Read-only cloud roles; encryption in transit/at rest; secrets in KMS/Secrets Manager.
Auditability	Every write captured with actor, before/after, timestamp, IP (or SSO claim).
Scalability	2k services, 500 clusters, 5k infra components, 20 concurrent users.
Cost Control	Retrieval-first; LLM summarization only when needed; token budgets & rate limits.

User Journeys

Journey A — SRE answers “what runs where” and fixes a mapping

1. Sign in with SSO → Chat UI.
2. Ask: “Where does **payments-api** run in prod?”
Bot returns clusters, namespaces, VPC, account, region, DB.
3. Add an association:
“Associate **payments-api** depends_on **arn:aws:rds:....db:pay-prod** in

prod.”

Bot validates, requests confirmation, writes association, and returns **audit id**.

4. **“Export payments-* service associations as CSV.”** → Download link.

Journey B — Platform engineer inventories clusters by env

1. Ask: **“List all EKS clusters in eu-west-1 and which services use them.”**
 2. Bot returns matrix of cluster↔services with owner team.
 3. **“Show JSON.”** → returns structured payload + download link.
-

APIs & Data Model

REST (sketch)

```
GET    /v1/discovery/services?name=payments-api&env=prod
GET    /v1/discovery/infra?type=eks_cluster&region=eu-west-1
GET    /v1/associations?service_id=svc-123&env=prod
POST   /v1/associations
PATCH /v1/associations/{id}
DELETE /v1/associations/{id}
POST   /v1/exports?format=csv&type=service-infra-map&env=prod
POST   /v1/admin/connectors/{provider}/refresh    # admin only,
rate-limited
```

Data Model

```
Service(id PK, name, repo_url?, owner_team?, business_unit?, tags
JSONB)
InfraComponent(id PK, type, cloud, account_id?, region?,
resource_arn?, k8s_uid?, name?, env?, tags JSONB)
Association(id PK, service_id FK, infra_id FK, relation_type, env,
created_by, created_at, updated_at, notes, confidence JSONB)
AuditLog(id PK, actor, action, target_type, target_id, before JSONB,
after JSONB, at timestamp)
```

Security, RBAC & Audit

- **Identity:** OIDC/SAML SSO → JWT in the app.
- **Roles:**
 - **Viewer:** discovery queries, read-only.
 - **Editor:** CRUD on associations.
 - **Admin:** connectors, RBAC, cache controls.
- **Audit:** All writes (including exports) are logged with full context.

Acceptance Criteria

- Discovery answers include referenced **accounts/regions** and **data sources**.
- Association **CRUD** works via Chat UI **and** API with RBAC & audit.
- Any visible table can be **exported** to CSV or JSON.
- **Read-only** cloud/K8s in place; no infra mutations beyond association DB.
- P95 latency $\leq 12s$ on cached queries.

Architecture Diagrams

11.1 User Interaction Sequence

```
sequenceDiagram
    autonumber
    actor User as SRE or Engineer
    participant UI as Chat UI (Web)
```

participant API as Backend API
participant RAG as Retrieval and Orchestrator
participant INV as Inventory Cache
participant CTX as Connectors AWS EKS CMDB
participant LLM as LLM Provider
participant DB as Assoc Store and Audit

User->>UI: Login via SSO
UI->>API: Exchange token (OIDC JWT)
User->>UI: "Where does payments-api run in prod?"
UI->>API: POST /chat {query}
API->>RAG: Normalize and plan query
RAG->>INV: Fetch cached inventory
alt Cache miss or forced refresh
 RAG->>CTX: Read-only describe calls
 CTX-->>RAG: Live inventory
 RAG->>INV: Update cache
end
RAG->>DB: Resolve associations for service env
RAG->>LLM: Summarize with grounded context
LLM-->>RAG: Answer draft
RAG-->>API: Structured answer plus table
API-->>UI: Render response

User->>UI: "Associate payments-api depends_on rds pay-prod"
UI->>API: POST /associations
API->>DB: Write association (RBAC check)
API->>DB: Write audit log
DB-->>API: OK
API-->>UI: Confirmation with association id

11.2 High-level Architecture

flowchart LR
 subgraph Client
 ChatUI["Chat UI (Web or Slack)"]
 end


```

subgraph Edge
  GW["API Gateway"]
  Auth["OIDC SSO"]
end

subgraph Backend
  Orchestrator["Query Orchestrator and RAG"]
  AssocAPI["Associations and Export API"]
  RBAC["RBAC Policy"]
  Cache["Inventory Cache"]
end

subgraph Data
  Store["Association Store and Audit Log"]
end

subgraph Providers
  LLM["LLM Provider"]
  AWS["AWS Read Only"]
  K8s["EKS or Kubernetes Read Only"]
  CMDB["CMDB Optional"]
  Obs["Observability Optional"]
end

ChatUI --> GW --> Auth --> GW
GW --> Orchestrator
GW --> AssocAPI
Orchestrator --> Cache
Orchestrator --> AWS
Orchestrator --> K8s
Orchestrator -.-> CMDB
Orchestrator -.-> Obs
Orchestrator --> LLM
AssocAPI --> Store
Orchestrator --> Store
RBAC --- GW
RBAC --- AssocAPI

```

11.3 Component-level Architecture

flowchart TD

subgraph Frontend

A1["Chat Composer"]

A2["Result Renderer tables and CSV"]

A3["Auth Client OIDC"]

end

subgraph Edge

B1["API Gateway"]

B2["AuthZ Filter JWT to RBAC"]

B3["Rate Limiter"]

end

subgraph Services

C1["Chat Controller"]

C2["Planner and Tools Router"]

C3["Retrieval Engine"]

C4["Connectors Service"]

C5["Association Service"]

C6["Export Service"]

C7["Audit Logger"]

end

subgraph Persistence

D1["Associations DB"]

D2["Exports Object Store"]

D3["Inventory Cache Redis"]

D4["Audit and Access Logs"]

end

subgraph External

E1["LLM API"]

E2["AWS SDK with STS AssumeRole"]

E3["EKS and Kubernetes API"]

E4["CMDB API"]

E5["Observability API"]

end

A1 --> B1 --> B2 --> C1
C1 --> C2
C2 --> C3
C3 --> D3
C3 -- cache miss or refresh --> C4
C4 --> E2
C4 --> E3
C4 -. optional .-> E4
C4 -. optional .-> E5
C3 --> C5
C2 --> E1
C5 --> D1
C6 --> D2
C5 --> C7 --> D4
A2 <-- responses --> C1
A3 --> B2

Test Plan

- **Unit:** entity validation, RBAC checks, connector mocks, export formatting.
 - **Integration:** chat → plan → cache → connectors → answer; CRUD → audit.
 - **Security:** read-only IAM policies; JWT tamper; role escalation attempts.
 - **Load:** 20 concurrent users; 5 QPS chat; token budget thresholds.
 - **Smoke:** fresh install, first-run connectors, first export.
-

Delivery & Ops

- Envs: **dev** → **test** → **stg** → **prod** (same IaC).
- Feature flags: **CMDB**, **Observability**, **write-ops**.

- Runbooks: connector health, cache warmers, token usage, audit review.
 - Metrics: request latency, cache hit rate, token usage, connector failure rate.
-

Epics & Stories for Jira

Below are **ready-to-paste** epics and stories. Use your project key (e.g., **SREBOT**). Estimated Story Points (SP) are indicative.

Epic 1 — Conversational Discovery

Goal: Answer inventory/topology/ownership queries with grounded data.

Acceptance: Queries return structured table + optional JSON/CSV.

Stories:

1. **Design query intents & prompts** — SP 3
 - AC: Glossary of intents; examples; negative prompts to avoid hallucination.
2. **Normalize & plan queries** — SP 5
 - AC: Planner selects data sources; falls back to cache; emits citations.
3. **Inventory query: EKS clusters by region/env** — SP 3
 - AC: Returns cluster list with account, region, env.
4. **Topology query: service → cluster/namespace** — SP 5
 - AC: Returns service deployments with cluster, ns, workload type.
5. **Ownership query via CMDB (optional)** — SP 3
 - AC: If CMDB connected, “owner team” field appears.
6. **“Show JSON” and “export CSV” paths** — SP 3
 - AC: Same data is serializable; export audited.

Epic 2 — Association Management (CRUD)

Goal: Create/update/delete service↔infra associations with validations and audit.

Stories:

1. **Association schema & migrations** — SP 3
 - AC: Tables created with indexes; sample seed data.
2. **Create association API & chat action** — SP 5
 - AC: Validates IDs; confirmation step; returns association id.
3. **Update association fields** — SP 3
 - AC: Update env/notes/relation_type; audit before/after.
4. **Delete association** — SP 3
 - AC: Soft delete or hard delete rule; audited.
5. **List associations by service/env** — SP 2
 - AC: Filterable endpoint; paginated.

Epic 3 — AWS & K8s Connectors + Cache

Goal: Reliable, cost-efficient read-only inventory.

Stories:

1. **AWS STS assume-role connector** — SP 5
 - AC: Read-only policies; rate-limited calls.
2. **EKS/K8s connector** — SP 5
 - AC: List clusters, namespaces, deployments, services.
3. **Cache service (Redis)** — SP 3
 - AC: 15-min TTL; cache-busting per account/cluster.

4. **On-demand refresh (admin)** — SP 3

- AC: Scoped refresh; observable; rate-limited.

5. **Connector health endpoints** — SP 2

- AC: Health with last successful sync timestamps.

Epic 4 — SSO, RBAC & Audit

Stories:

1. **SSO integration (OIDC/SAML)** — SP 5

- AC: JWT available to backend; roles mapped via claims or lookup.

2. **RBAC middleware** — SP 3

- AC: Viewer/Editor/Admin enforced on every route.

3. **Audit logging service** — SP 3

- AC: All writes (CRUD + export) are recorded.

4. **Admin role & feature flags** — SP 2

- AC: Toggle CMDB/Obs/write-ops; visible in config endpoint.

Epic 5 — Exports

Stories:

1. **Export CSV/JSON from any table** — SP 3

- AC: Filename scheme, size limits, 1-click download.

2. **S3/object storage for exports** — SP 3

- AC: Signed URL; expiry; audit entry.

Epic 6 — UI/UX Chat

Stories:

1. **Chat composer + history** — SP 3
 - AC: Persisted history; copy to clipboard.
2. **Table renderer + “export” button** — SP 3
 - AC: Sortable columns; download triggers export service.
3. **Confirmation UI for write actions** — SP 2
 - AC: Shows diff before/after; cancel path.

Epic 7 — Observability & CMDB (Optional R1)

Stories:

1. **CMDB connector read-only** — SP 3
2. **Observability metadata** — SP 3
 - Both behind feature flags; optional for R1 acceptance.*

Epic 8 — Non-Functional Hardening

Stories:

1. **Perf budget tests** — SP 3
2. **Security posture review (IAM, secrets)** — SP 3
3. **Error handling & user-facing messages** — SP 2
4. **Rate limiting & token budget controls** — SP 2
5. **Logging & tracing with correlation IDs** — SP 3

Definition of Ready (DoR): user story has goal, AC, test notes, dependencies, mock data.

Definition of Done (DoD): code merged, tests pass, docs updated, observability in place, demo recorded.

How to post Epics & Stories using a Jira MCP Server

The steps below assume you have an MCP-compatible client (e.g., an agent runner or a desktop client) and a **Jira MCP server** that exposes Jira operations (list projects, get create meta, create issues, update issues, etc.). Exact tool names vary by server—first list tools and adjust accordingly.

1) Configure the Jira MCP server

1. Install or run the Jira MCP server (container or binary).
2. Set environment/config (examples):
 - `JIRA_BASE_URL=https://your-domain.atlassian.net`
 - `JIRA_EMAIL=you@company.com`
 - `JIRA_API_TOKEN=<jira-personal-api-token>`
 - `JIRA_PROJECT_KEY=SREBOT` (*replace with your Jira project key*)
3. Start the server and confirm it registers tools.

2) Discover available tools & fields

- Ask your MCP client to **list tools** exposed by the Jira server (e.g., `jira.list_tools`).
- Call the tool for **create metadata** (often named like `jira.get_create_meta` or `issues.getCreateMeta`) to determine:
 - Valid **issue types** (`Epic`, `Story`, `Task`).
 - Required **fields** for each type.
 - The field ID for **Epic Name** (in many Jira Cloud projects this is `customfield_10011`, but **do not assume**—read from meta).

3) Create a “Release 1” Fix Version (optional but recommended)

- Use a tool such as `jira.create_version` with:
 - `project: SREBOT`
 - `name: Release 1`
 - `description: SRE Bot – Infrastructure & Platform discovery`
 - `released: false`

4) Create the Epics

For each epic above, call the server's create-issue tool (e.g., `jira.create_issue`). Supply:

- `project: SREBOT`
- `issuetype: Epic`
- `summary: short Epic title (e.g., "Conversational Discovery")`
- `description: copy the Epic block from this doc`
- `labels: ["srebot", "release1"]`
- `fixVersions: ["Release 1"]`
- `Epic Name` field (use the **field id** discovered in step 2)

Example payload (generic JSON shown, adapt to your MCP tool shape):

```
{
  "project": "SREBOT",
  "issuetype": "Epic",
  "summary": "Conversational Discovery",
  "description": "Answer inventory/topology/ownership queries...",
  "labels": ["srebot", "release1"],
  "fixVersions": ["Release 1"],
  "fields": {
```

```
    "customfield_EPIC_NAME": "Conversational Discovery"
  }
}
```

Replace `customfield_EPIC_NAME` with the actual field id returned by `get_create_meta`.

Record the created **Epic keys** (e.g., `SREBOT-101`, `SREBOT-102`, ...).

5) Create Stories under each Epic

For each story:

- `project`: `SREBOT`
- `issuetype`: `Story`
- `summary`: from the story list (e.g., "Inventory query: EKS clusters by region/env")
- `description`: include Acceptance Criteria and test notes from this doc
- `labels`: `["srebot", "release1"]`
- `fixVersions`: `["Release 1"]`
- **Link the story to its Epic:**
 - On many Jira MCP servers, set the **Epic link field** (classic projects) or **parent** (team-managed).
 - Use **create meta** to confirm which linkage field is required:
 - *Classic*: a custom field often named `Epic Link` (id varies, e.g., `customfield_10014`).
 - *Team-managed*: `parent` can reference the Epic ID.
 - Provide the **Epic issue id/key** accordingly.

Example story payload (classic with Epic Link):

```
{
  "project": "SREBOT",
  "issuetype": "Story",
  "summary": "Topology query: service to cluster and namespace",
  "description": "AC: Returns service deployments with cluster,
namespace, workload type.",
  "labels": ["srebot", "release1"],
  "fixVersions": ["Release 1"],
  "fields": {
    "customfield_EPIC_LINK": "SREBOT-101"
  }
}
```

Example story payload (team-managed using parent):

```
{
  "project": "SREBOT",
  "issuetype": "Story",
  "summary": "Association schema and migrations",
  "description": "AC: Tables created with indexes; sample seed data.",
  "labels": ["srebot", "release1"],
  "fixVersions": ["Release 1"],
  "parent": { "key": "SREBOT-102" }
}
```

6) (Optional) Bulk creation

- If the Jira MCP server exposes `bulk_create_issues`, send an array of issue payloads.
- Otherwise, loop over the epics first, store epic keys, then create stories referencing those keys.

7) Verify and tidy up

- Use search tools (e.g., `jira.search_issues`) to list `project=SREBOT AND fixVersion="Release 1" ORDER BY type, priority`.

- Add components, assignees, and sprint if desired (via `update_issue`).
- Transition to “Ready for Dev” based on your workflow, or simply leave in backlog.

8) Common troubleshooting tips

- **Epic Name is required** for Epic creation in many templates—confirm the field id.
 - If **Epic Link** fails, you’re likely in a team-managed project; use `parent`.
 - If validation fails, call **create meta** again and compare required fields.
-

Ready-to-use Issue Seeds (copy/paste summaries)

Epics

- Conversational Discovery
- Association Management (CRUD)
- AWS & K8s Connectors + Cache
- SSO, RBAC & Audit
- Exports
- UI/UX Chat
- Observability & CMDB (Optional R1)
- Non-Functional Hardening

Stories — use the lists in Epics & Stories for Jira.